Relazione Progetto Connections

Reti e Laboratorio III - A.A. 2025/26
Filippo Pacini, appello di Marzo.

1. Scelte progettuali

Libreria JSON: Gson 2.10.1.

Thread Pool: ThreadPoolExecutor custom con corePoolSize=10, maxPoolSize=30, coda bounded di capacità 50. I 10 core thread restano sempre attivi per il carico base; sotto pressione il pool scala fino a 30, con i thread extra distrutti dopo 60s di inattività. La coda bounded evita accumulo infinito. La CallerRunsPolicy gestisce il sovraccarico: se pool e coda sono pieni, il thread chiamante (accept) esegue il task, creando backpressure naturale.

Timer partite: ScheduledExecutorService. Un singolo thread schedula la fine della partita dopo la durata configurata. Preferito questo per esempio rispetto a Thread.sleep() poiché non blocca un thread per l'intera durata.

Persistenza ibrida. Salvataggio periodico ogni 30 secondi (configurabile) tramite ScheduledExecutorService, più salvataggio forzato a fine di ogni partita e allo shutdown del server (via shutdown hook). La scrittura usa la tecnica temp-file + rename atomico per evitare corruzione in caso di crash.

Classifica cached. La classifica viene ricalcolata (sort) solo alla fine di ogni partita e servita dalla cache per le richieste. Con poche centinaia di utenti il costo del sort si considera trascurabile, e le letture sono immediate.

Protocollo di framing TCP. Ogni messaggio JSON è preceduto da 4 byte (int big-endian) che indicano la lunghezza del payload. Questo permette al ricevente di sapere esattamente quanti byte leggere, gestendo correttamente i partial read/write tipici di NIO non-blocking.

Notifiche UDP. Fine partita e inizio nuova partita sono notificate via UDP unicast a ogni client loggato. Se un pacchetto si perde, il client si riallinea alla prossima richiesta TCP. Il client comunica la propria porta UDP al login. La porta UDP viene assegnata automaticamente dal sistema operativo (porta 0 nel file di configurazione), permettendo di lanciare più client sulla stessa macchina senza conflitti.

Parametri di configurazione. Tutti i parametri (porte, durata partite, dimensioni pool, intervallo persistenza) sono letti da file .properties separati per client e server.

2. Schema dei thread

2.1 Server - 4 gruppi di thread

Main (1 thread) - Accept TCP: esegue un loop su ServerSocket.accept() e assegna ogni nuova connessione al ThreadPoolExecutor.

Worker Pool (10-30 thread) - Gestione client: ogni connessione è gestita da un ConnectionHandler che legge messaggi JSON, li delega al RequestHandler, e invia la risposta.

GameScheduler (1 thread) - Timer partite: ScheduledExecutorService che schedula la callback onGameTimeUp() alla scadenza della durata della partita. Gestisce la transizione tra partite successive.

PersistenceScheduler (1 thread) - Salvataggio periodico: ScheduledExecutorService che esegue saveAll() ogni 30 secondi. Thread daemon, non blocca lo shutdown della JVM.

2.2 Client - 3 thread

Main / CLI (1 thread) - Interfaccia utente: loop che legge comandi da stdin, li converte in JSON, li mette nella coda NIO e attende la risposta.

NIO Selector (1 thread) - I/O TCP non-blocking: loop su Selector che gestisce OP_READ e OP_WRITE. Comunica col Main tramite codice thread-safe.

UDP Listener (1 thread) - Notifiche asincrone: loop bloccante su DatagramSocket.receive(). Stampa le notifiche (gameEnd, newGame) direttamente in console.

Comunicazione inter-thread nel client: il Main e il NIO Selector comunicano tramite due LinkedBlockingQueue: una per le richieste (Main -> Selector) e una per le risposte (Selector -> Main). Il thread UDP è completamente indipendente.

3. Strutture dati

3.1 Lato server

Utenti registrati - ConcurrentHashMap<String, UserRecord> con chiave username. Lookup, lock per segmento, alta concorrenza su registrazione e login.

Sessioni attive - ConcurrentHashMap<String, SessionInfo> con chiave username. Verifica login, iterazione per broadcast UDP.

Stato giocatore nella partita - ConcurrentHashMap<String, PlayerGameState> con chiave username, contenuta in ogni oggetto Game. Aggiunta concorrente di giocatori durante la partita.

Storico partite - ConcurrentHashMap<Integer, Game> con chiave gameId. Lookup per requestGameInfo e requestGameStats.

Classifica (cache) - List<RankingEntry> protetta da ReentrantReadWriteLock. Ricalcolata una sola volta a fine partita.

Gruppi di una partita - List<GameGroup> immutabile. Creata alla lettura del JSON, mai modificata.

Parole mescolate - List<String> immutabile. Collections.shuffle() alla creazione, poi Collections.unmodifiableList().

3.2 Lato client

Il client non mantiene strutture dati persistenti. Lo stato locale comprende lo username corrente (stringa), le due code NIO (LinkedBlockingQueue<byte[]> per le richieste e LinkedBlockingQueue<String> per le risposte), e i buffer NIO per il framing dei messaggi TCP.

4. Primitive di sincronizzazione

ConcurrentHashMap (utenti, sessioni, stati giocatore, storico) - Lock interno per segmento/bucket. Non serve sincronizzazione esplicita: putIfAbsent() e computeIfAbsent() sono atomici. Più thread possono accedere a segmenti diversi in parallelo.

PlayerGameState - synchronized(this), lock fine-grained per giocatore. Ogni thread fa lock solo lo stato del proprio giocatore, permettendo a N giocatori di giocare in parallelo senza contesa. Protegge la sequenza: valuta proposta -> aggiorna errori -> verifica sconfitta.

UserRecord (statistiche) - synchronized(this). Aggiornamento atomico delle statistiche a fine partita (score, streak, histogram).

Classifica cached - ReentrantReadWriteLock. readLock per le consultazioni concorrenti (frequenti), writeLock solo durante il ricalcolo (raro, una volta per partita).

Code NIO client - LinkedBlockingQueue, thread-safe. offer() dal Main, poll() dal Selector e viceversa.

Game.active - volatile boolean. Visibilità garantita tra thread senza lock. Il GameScheduler lo setta a false, i worker lo leggono per verificare se la partita è ancora in corso.

sendMessage (TCP) - synchronized. Protegge la scrittura sulla socket TCP da accessi concorrenti.

Strategia generale: si "privilegia" per così dire la granularità fine dei lock per massimizzare il parallelismo. Le ConcurrentHashMap evitano lock globali; il synchronized su PlayerGameState permette a N giocatori di giocare in parallelo senza contesa; il ReadWriteLock sulla classifica permette letture concorrenti illimitate.

5. Compilazione, esecuzione e comandi

Posizionarsi nella cartella "consegna".

5.1 Compilazione e creazione JAR

Su Windows PowerShell:

```
.\build.bat
```

Su Linux/macOS:

```
./build.sh
```

5.2 Esecuzione

Avvio del server:

```
java -jar ConnectionsServer.jar
```

Avvio del client (in un terminale separato):

```
java -jar ConnectionsClient.jar
```

La porta UDP viene assegnata automaticamente dal sistema operativo, quindi è possibile lanciare più client sulla stessa macchina senza modificare nulla.

5.3 Comandi del client *(sono visibili con il comando "help")*

register <user> <pwd> - Registra un nuovo account.

login <user> <pwd> - Effettua il login e riceve le parole della partita corrente.

logout - Effettua il logout.

update-credentials - Modifica username e/o password (interattivo).

propose <w1> <w2> <w3> <w4> - Invia una proposta di 4 parole. Per parole composte usare le parentesi, es: propose (ICE CUBE) W2 W3 W4.

game-info [gameId | current] - Stato o esito di una partita.

game-stats [gameId | current] - Statistiche di una partita.

leaderboard all | top K | player N - Classifica.

my-stats - Statistiche personali del giocatore.

quit - Esci dal gioco.

5.4 Parametri di configurazione

Tutti i parametri (porte TCP/UDP, dimensioni thread pool, durata partita, intervallo persistenza, percorsi file) sono configurabili nei file server.properties e client.properties.